

Proyecto de PruebaJR

Este proyecto es una aplicación web de gestión de publicaciones desarrollada con Next.js y Axios. Permite a los usuarios crear, editar, ver y eliminar publicaciones, comentarios y álbumes. La interfaz es interactiva y utiliza tarjetas para mostrar los elementos, y los usuarios pueden alternar entre ver un número limitado de publicaciones, comentarios o álbumes, o todos disponibles. Los datos provienen de una base de datos pública en [Neon Serverless Postgres](#). Además, incluye formularios para agregar nuevos elementos y la capacidad de eliminar los existentes.

Características

- **Lista de publicaciones:** Muestra una lista de publicaciones obtenidas de la base de datos.
- **Crear nueva publicación:** Permite a los usuarios agregar nuevas publicaciones mediante un formulario.
- **Editar publicaciones:** Los usuarios pueden editar el título y el contenido de las publicaciones existentes.
- **Eliminar publicaciones:** Los usuarios pueden eliminar publicaciones de las publicaciones existentes.
- **Visibilidad de publicaciones:** Los usuarios pueden alternar entre ver solo las primeras 4 publicaciones o todas las disponibles.
- **Manejo de errores:** Se muestra un mensaje de error si ocurre un problema al cargar, agregar o editar publicaciones.
- **Interfaz interactiva:** Utiliza botones de acción y una interfaz de usuario basada en tarjetas.
- **Lista de comentarios:** Muestra una lista de comentarios obtenidas de la base de datos y filtradas por el id de la publicación.
- **Crear nuevo comentario:** Permite a los usuarios agregar nuevos comentarios mediante un formulario.
- **Eliminar comentarios:** Los usuarios pueden eliminar comentarios de los comentarios existentes.
- **Lista de albums:** Muestra una lista de albums obtenidas de la base de datos y filtradas por el id de la publicación.
- **Crear nuevos albums:** Permite a los usuarios agregar nuevos albums mediante un formulario.
- **Eliminar albums:** Los usuarios pueden eliminar albums de los albums existentes.

Instalación

1. Clona el repositorio:

```
git clone https://github.com/Saimon1520/prueba-jr.git
```

2. Navega a la carpeta del proyecto:

```
cd prueba-jr
```

3. Instala las dependencias:

```
npm install
```

4. Ejecuta el proyecto:

```
npm run dev
```

5. Abre la aplicación en tu navegador: <http://localhost:3000>

Estructura del Proyecto

App

- **src/app/layout.tsx:** Componente principal que maneja la estructura global del sitio, incluyendo la barra de navegación y el diseño base.
- **src/app/page.tsx:** Página de inicio donde se muestra un mensaje de bienvenida y el formulario para agregar publicaciones.

Albums

- **src/app/albums/page.tsx:** Página que permite mostrar una lista de albums, agregar o editar albums mediante un formulario.

Api

- **src/app/api/album/route.ts:** La ruta que permite agregar, editar, obtener y eliminar cada album.
- **src/app/api/comment/route.ts:** La ruta que permite agregar, editar, obtener y eliminar cada comentario.
- **src/app/api/login/route.ts:** La ruta que permite validar que un usuario este registrado validando su email y contraseña.
- **src/app/api/posts/route.ts:** La ruta que permite agregar, editar, obtener y eliminar cada publicación.
- **src/app/api/users/route.ts:** La ruta que permite agregar, editar, obtener y eliminar cada usuario.

Comments

- **src/app/comments/[id]/page.tsx:** Página que muestra una lista de comentarios con la opción eliminar cada uno y agregar comentarios.

Login-Form

- **src/app/login-form/page.tsx:** Página que permite validar el correo electrónico y la contraseña mediante un formulario.

Post-Form

- **src/app/post-form/page.tsx:** Página que permite agregar publicaciones mediante un formulario.

Posts

- **src/app/posts/page.tsx**: Página que muestra una lista de publicaciones con la opción de editar y eliminar cada una.

Register

- **src/app/register/page.tsx**: Página que permite agregar usuarios mediante un formulario.

Components

- **src/app/components/Navbar.tsx**: Componente de navegación que permite la navegación entre las páginas principales del sitio.
- **src/app/components/PostForm.tsx**: Formulario para agregar o editar publicaciones.
- **src/app/components/UserForm.tsx**: Formulario para agregar usuarios.

Context

- **src/app/context/AlbumContext.tsx**: Contexto global que maneja el estado de los album y proporciona funciones para agregar, editar, eliminar y alternar la visibilidad de los album.
- **src/app/context/CommentContext.tsx**: Contexto global que maneja el estado de los comentarios y proporciona la funcion de eliminar y agregar los comentarios.
- **src/app/context/LoginContext.tsx**: Contexto global que maneja el estado del usuario si esta logueado o no.
- **src/app/context/PostContext.tsx**: Contexto global que maneja el estado de las publicaciones y proporciona funciones para agregar, editar y alternar la visibilidad de las publicaciones.
- **src/app/context/UserContext.tsx**: Contexto global proporciona funciones para agregar usuarios.

Lib

- **src/app/lib/prisma.ts**: Crea el Prisma Client.

Styles

- **src/app/styles/globals.css**: Estilos globales de la aplicación, incluyendo soporte para un modo oscuro.

/

- **tsconfig.json**: Configuración de TypeScript.
- **package.json**: Dependencias y scripts del proyecto.

Funcionalidades Detalladas

Cargar Publicaciones

- Las publicaciones se cargan al inicio mediante una solicitud GET a una base de datos de Neon Serverless Postgres.

Agregar Publicaciones

- Los usuarios pueden agregar nuevas publicaciones mediante el formulario en el apartado Crear Publicación.

Editar Publicaciones

- Los usuarios pueden editar una publicación haciendo clic en el botón "Editar" en cada tarjeta de publicación.

Alternar Visibilidad de Publicaciones

- Los usuarios pueden alternar entre ver solo las primeras 4 publicaciones o todas las disponibles con el botón "Show More" o "Show Less".

Cargar Comentarios

- Los comentarios se cargan al inicio mediante una solicitud GET a una base de datos de Neon Serverless Postgres.

Agregar Comentarios

- Los usuarios pueden agregar nuevos comentarios a través del formulario disponible en la sección de comentarios, al hacer clic en "Ver comentarios" dentro de una publicación en el apartado de publicaciones.

Eliminar Comentarios

- Los usuarios pueden eliminar un comentario haciendo clic en el botón "Eliminar" en cada tarjeta de comentario.

Alternar Visibilidad de Comentarios

- Los usuarios pueden alternar entre ver solo los primeros 4 comentarios o todos los disponibles con el botón "Show More" o "Show Less".

Cargar Albums

- Los albums se cargan al inicio mediante una solicitud GET a una base de datos de Neon Serverless Postgres.

Agregar Albums

- Los usuarios pueden agregar nuevos albums a través del formulario disponible en la sección de albums.

Eliminar Albums

- Los usuarios pueden eliminar un album haciendo clic en el botón "Eliminar" en cada tarjeta de album.

Alternar Visibilidad de Albums

- Los usuarios pueden alternar entre ver solo los primeros 4 albums o todos los disponibles con el botón "Show More" o "Show Less".

Dependencias

- **Next.js:** Framework de React para aplicaciones de servidor.

- **React:** Librería para construir interfaces de usuario.
- **Axios:** Cliente HTTP para realizar solicitudes.
- **Tailwind CSS:** Framework de CSS para estilos rápidos y responsivos.
- **NextUI:** NextUI es una biblioteca de componentes UI para React, que facilita crear interfaces modernas.

Desarrollo

- **Instalación de dependencias:**

```
npm install
```

- **Ejecutar el proyecto:**

```
npm run dev
```

- **Linter:** El proyecto usa ESLint para el análisis de código.

```
npm run lint
```

Desarrollo

El proyecto se puede probar en [Prueba-Jr](#).

Recomendaciones Propuestas

1. Incorporar una seccion dentro de cada album

Recomiendo agregar una seccion dentro de cada album para agregar imagenes o links de musica.

2. Incorporar la sección de tareas

Recomiendo agregar la sección de tareas ya que estaba disponible en JSONPlaceholder.

Decisiones Técnicas Tomadas

1. Decidí crear un contexto para las publicaciones y otro para los comentarios debido a que la API no permite realizar cambios reales en los datos. Esto me permite manejar localmente los cambios realizados, como editar, agregar o eliminar publicaciones, y eliminar comentarios. Sin embargo, mantuve las consultas a la API, ya que comprendí que era importante seguir consumiéndola como parte del proyecto.
2. Opté por implementar un navbar para facilitar la navegación dentro de la página web. Esto también sirve como una representación de cómo podría extenderse la página en el futuro, permitiendo agregar más apartados según sea necesario.
3. Elegí usar la licencia MIT, ya que planeo hacer público el proyecto una vez que sea revisado por la empresa que actualmente me está evaluando.

4. Decidí utilizar Axios como cliente HTTP porque ofrece una mejor gestión de errores (como los códigos 404, 400 y 500), es compatible con navegadores antiguos y maneja automáticamente las respuestas en formato JSON, lo que simplifica el desarrollo.
5. Decidí utilizar NextUi para que me ayudara con el diseño de la página.

Licencia

Este proyecto está bajo la Licencia MIT. Consulta el archivo LICENSE.txt para más detalles.